Language

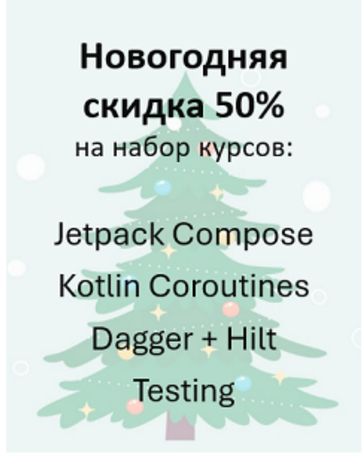
Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 10. Время жизни компонентов

21 июня 2021

В этом уроке обсудим время жизни компонентов.

В уроке про сабкомпоненты я упоминал, что они дают нам более гибкое время жизни объектов, чем один компонент. В этом участвует механизм, который называется scope.

Scope - это достаточно трудная для объяснения тема. Я начал было писать урок по ней по обычной схеме, когда для объяснения используется аннотация Singleton. Но в итоге понял, что получается непонятно. Так что попробуем зайти с другой стороны и поговорим сначала про такое понятие, как время жизни.

Мы создаем компонент или сабкомпонент, и он живет и работает для нас некоторое время. Но в какой-то момент он становится нам не нужен, и он уничтожается. А когда он снова нам понадобится, мы создаем новый экземпляр этого компонента.

Давайте на примерах рассмотрим, когда и кем компонент создается, уничтожается и снова пересоздается.

В качестве первого примера возьмем компонент AppComponent, который мы уже рассматривали в прошлых примерах.

А вторым примером будет его сабкомпонент - OrderComponent, который используется в OrderActivity. Он полностью аналогичен сабкомпоненту MainComponent, который мы использовали в MainActivity в прошлых примерах.

Почему я в примере сменил MainActivity на OrderActivity? Разница в том, что MainActivity обычно является стартовым и живет все время работы приложения. На нем не показать нормальный жизненный цикл Activity. А нам для хорошего примера нужно Activity, которое пользователь открывает и закрывает несколько раз в течение использования приложения.

Будем считать, что OrderActivity отображает детали заказа в нашем приложении. Пока пользователь находится в приложении, он смотрит детали разных заказов, тем самым открывая и закрывая экран OrderActivity.

Компонент

Вспоминаем, как мы обычно получаем AppComponent:

```
class App : Application() {

    val appComponent: AppComponent = DaggerAppComponent.create()

    // ...

}
```

Мы создаем его в момент создания Application класса.

Когда он будет уничтожен? Когда мы закрываем приложение. Как только система уничтожит объект Application, сборщик мусора уничтожит и наш компонент.

Когда мы в следующий раз запустим приложение, новый экземпляр Application создаст новый экземпляр компонента AppComponent.

Т.е. AppComponent жив пока живет приложение. При перезапуске приложения мы получаем новый экземпляр компонента.

Сабкомпонент

Теперь давайте посмотрим на сабкомпонент. Он начинает жить при старте Activity:

```
class OrderActivity : AppCompatActivity() {

    lateinit var orderComponent: OrderComponent

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_order)
        orderComponent = (application as App).appComponent.getOrderComponent()

        // ...
    }

}
```

Мы из application берем компонент AppComponent и вызываем его метод getOrderComponent. Компонент создает для нас сабкомпонент OrderComponent.

Когда этот сабкомпонент будет уничтожен? Когда мы закроем OrderActivity. Как только система уничтожит объект OrderActivity, сборщик мусора уничтожит и наш сабкомпонент.

Когда мы в следующий раз откроем этот экран, мы снова вызовем метод getOrderComponent и компонент AppComponent создаст нам новый экземпляр сабкомпонента OrderComponent.

Т.е. сабкомпонент OrderComponent жив, пока живет OrderActivity. При каждом новом запуске этого экрана мы получаем новый сабкомпонент.

Давайте проговорим несколько важных моментов.

1) Обратите внимание, что компонент AppComponent продолжит жить после закрытия OrderActivity. Он ведь хранится в Application объекте и не привязан к жизни никакого Activity.

Когда мы в следующий раз запустим экран OrderActivity, и из Application объекта возьмем AppComponent, это будет ровно тот же компонент, что и в прошлый раз.

2) Каждый раз вызывая у компонента метод getOrderComponent, мы будем получать новый сабкомпонент OrderComponent. Компонент не держит у себя ссылку на сабкомпонент и не управляет им. Он его создает, отдает нам и забывает про него.

Таким образом, задача компонента - только создать сабкомпонент (и предоставить ему в пользование свои модули). А время жизни сабкомпонента определяем мы. Мы храним этот сабкомпонент сколько нам нужно, а потом отпускаем, чтобы сборщик мусора его уничтожил.

Никакого ручного уничтожения компонентов или сабкомпонентов не выполняется.

3) На этих двух примерах видно, что время жизни сабкомпонента меньше, чем время жизни компонента. За время жизни компонента (время работы приложения) проходит несколько жизненных циклов сабкомпонента (открытие/закрытие экрана).

4) Мы в Activity.onCreate вызываем код получения сабкомпонента после вызова super.onCreate. Но это можно делать и до. В Уроке 14 мы рассмотрим случай передачи сабкомпонента от Activity фрагменту. В этом случае сабкомпонент необходимо получить до вызова super.onCreate.

Назад Вперёд

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.